

STŘEDOŠKOLSKÁ ODBORNÁ ČINNOST

Obor: Informatika

Mobilní aplikace pro robotické rameno

Smartphone application for robotic arm

Autor: Šimon Sýkora

Škola: SPŠ Žďár nad Sázavou, Studentská 761, 591 01

Kraj: Vysočina

Konzultant: Ing. Jaroslav Příbramský

Rok: 2026

Prohlášení

Prohlašuji, že jsem svou práci SOČ vypracoval samostatně a použil jsem pouze prameny a literaturu uvedené v seznamu bibliografických záznamů.

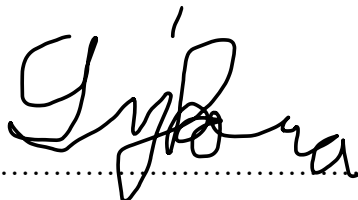
Beru na vědomí, že nejpozději odevzdáním slovesné vědecké práce do veřejné soutěže Středoškolská odborná činnost, stejně jako odevzdáním jejích příloh a dalších připojených děl, např. audiovizuálních, fotografických, výtvarných, architektonických apod. (dále jen „soutěžní dílo“), dochází ke zveřejnění díla podle § 4 odst. 1 zákona č. 121/2000 Sb., autorského zákona, ve znění pozdějších předpisů (dále jen „autorský zákon“). Totéž platí pro pozdější odevzdání doplněného, změněného, upraveného nebo opraveného díla.

Beru na vědomí, že zveřejněním díla, jehož součástí je vynález, se tento vynález stává součástí stavu techniky podle § 5 odst. 1, 2 zákona č. 527/1990 Sb., o vynálezech, průmyslových vzorech a zlepšovacích návrzích, ve znění pozdějších předpisů (dále jen „patentový zákon“), což zakládá překážku pro udělení patentu podle § 3 odst. 1 patentového zákona.

Beru na vědomí, že vyhlášovatel soutěže je podle § 61 odst. 1 autorského zákona per analogiam oprávněn užít soutěžní dílo pro účely zajištění průběhu soutěže, zejména k zajištění transparentnosti soutěže a veřejnosti obhajob soutěžních prací. V odůvodněném rozsahu je tedy vyhlášovatel po dobu účasti autora v soutěži oprávněn zejména:

- zhotovovat rozmnoženiny díla, je-li to nezbytné k seznámení účastníků soutěže, porotců nebo veřejnosti se soutěžní prací;
- zapůjčit originál nebo rozmnoženinu díla účastníkům soutěže, porotcům nebo veřejnosti. Přitom dbá na bezpečné nakládání s dílem;
- vystavovat originál nebo rozmnoženinu díla v průběhu soutěžních přehlídek a doprovodných akcí;
- sdělovat dílo veřejnosti v nemotné podobě, a to především počítačovou nebo obdobnou sítí.

Dále prohlašuji, že při tvorbě této práce jsem použil nástroj generativního modelu AI Github Copilot; <https://github.com/copilot>, Chat GPT, <https://chatgpt.com/>, za účelem kontroly gramatiky, úpravy textu, reformulaci textu, pomoc se strukturou, pomoc konkrétně v projektu s vytvořením složitých funkcí, správného nakonfigurování prostředí, ošetření chyb, vysvětlení různých problémů. Po použití tohoto nástroje jsem provedl kontrolu obsahu a přebírám za něj plnou zodpovědnost.



Ve Žďáře nad Sázavou dne 12.3.....
Šimon Sýkora

Anotace (nebo Souhrn)

Tato práce se zabývá návrhem a realizací mobilní aplikace pro ovládání robotického ramene pomocí mobilního telefonu. Hlavním cílem je vytvořit funkční uživatelsky přístupnou aplikaci, která umožní uživateli bezdrátově řídit pohyby jednotlivých částí stupňů volnosti konkrétního, výchozího, volně dostupného robotického ramene a vytvoření a spuštění vlastního automatizovaného pohybu (sekvenci). Počátkem projektu bylo vybrání vhodného robotického ramene, jeho vytisknutí, sestavení, realizaci zapojení jednotlivých komponent a vytvoření firmwaru. Následně tedy vytvoření softwaru jako aplikaci pro mobilní zařízení. Řešení využívá mikrokontrolér Arduino a Bluetooth komunikaci, přičemž mobilní aplikace je vytvořena ve frameworku Flutter. V rámci práce je vytvořena logika přenosu řídicích příkazů, zpracování dat na straně zařízení a samotné řízení servomotorů robotického ramene. Výsledkem je funkční prototyp umožňující ovládání pěti os robotické ruky, jak manuálně, tak automaticky, přes bezdrátové ovládání mobilní aplikace. Přínosem práce je ukázka praktického využití mobilní aplikace pro řízení mechanického systému a vytvoření základu pro další rozšíření.

Klíčová slova

Robotické rameno; Arduino; Bluetooth; Flutter; mobilní aplikace; bezdrátové ovládání; servomotory; řízení pohybu;

Annotation (or Summary)

This thesis deals with the design and implementation of a mobile application for controlling a robotic arm using a mobile phone. The main goal is to create a functional, user-friendly application that allows the user to wirelessly control the movements of individual degrees of freedom of a specific, freely available robotic arm and to create and run their own automated movement (sequence). The project began with the selection of a suitable robotic arm, its printing, assembly, implementation of the connection of individual components, and creation of firmware. This was followed by the creation of software as an application for mobile devices. The solution uses an Arduino microcontroller and Bluetooth communication, with the mobile application created in the Flutter framework. The work includes the creation of logic for the transmission of control commands, data processing on the device side, and the actual control of the servomotors of the robotic arm. The result is a functional application that allows the five axes of the robotic arm to be controlled both manually and automatically via

wireless control of the mobile application. The contribution of the work is to demonstrate the practical use of a mobile application for controlling a mechanical system and to create a basis for further expansion.

Keywords

Robotic arm; Arduino; Bluetooth; Flutter; mobile application; wireless control; servomotors; motion control;

Obsah

1	Úvod	7
2	Teoretická část	9
2.1	Robotická ramena a jejich využití	9
2.2	Konstrukce a řízení robotického ramene	9
2.3	Mikrokontrolery v robotice se zaměřením na Arduino	10
2.4	Servomotory a jejich použití v robotických mechanismech	10
2.5	Bezdrátová komunikace Bluetooth při řízení zařízení	11
2.6	Mobilní aplikace pro řízení technických systémů	12
2.7	Framework Flutter a jeho využití v projektu	12
2.8	Automatizované sekvence pohybů	12
3	Cíle práce	13
4	Vlastní projekt	14
4.1	Konstrukční řešení	14
4.1.1	Důvod výběru původního modelu robotického ramene	14
4.1.2	Původní způsob ovládání a jeho omezení	14
4.1.3	Změna konstrukční koncepce – ovládání pomocí Bluetooth	14
4.1.4	Mechanická konstrukce a stupně volnosti	15
4.1.5	Konstrukční řešení zápěstí a tvorba	16
4.1.6	Elektronika a napájení	16
4.2	Vývojové nástroje pro tvorbu mobilní aplikace	17
4.2.1	Zdůvodnění volby frameworku Flutter a jazyka Dart	17
4.2.2	Vývojové prostředí pro mobilní aplikaci	18

4.2.3	Android Studio a vývojové závislosti	18
4.2.4	Konfigurace vývojového prostředí	18
4.2.5	Debugování a testování aplikace	19
4.2.6	Vývojové nástroje pro programování robota	19
4.3	Návrh interface obslužné aplikace	20
4.3.1	Účel aplikace a cílový uživatel	20
4.3.2	Použité technologie a principy UI	20
4.3.3	Navigace a ovládací prvky	22
4.3.4	Domovská obrazovka – sken Bluetooth zařízení	22
4.4	Integrace Bluetooth	24
4.5	Obrazovka s manuálním ovládáním	24
4.5.1	Bezpečnostní zámky ovládání	25
4.6	Sekvence.....	27
4.6.1	Formát souboru sekvence	28
4.6.2	Ukázka kódu sekvence	29
4.6.3	UI sekvence	29
5	Obrazovka s nastavením	30
6	Diskuze	32
7	Závěr	34
8	Použitá literatura	Chyba! Záložka není definována.
9	Seznam použitých zkratk	36
10	Seznam obrázků	37

1 Úvod

V současné době se robotické systémy stále uplatňují nejen v průmyslu, ale také ve výuce, domácích projektech a zájmové technické činnosti. Rozvoj dostupných mikrokontrolerů, mobilních zařízení a vývojových nástrojů umožňuje vytvářet řešení, která byla dříve vyhrazena především profesionálním pracovištím. Právě propojení mechanické konstrukce, elektroniky a softwaru dnes představuje za mě jednu z nejzajímavějších technických oblastí, které vedou ke kreativnímu a logickému vytvoření systémů či aplikací schopných reagovat na požadavky uživatele. Navíc mně vždy bavilo vytvářet design a layout uživatelských rozhraní, sestavovat různé zařízení, pracovat s Arduinem a rozvíjet se v programování, a proto jsem se rozhodl vytvořit systém pro ovládání robotického ramene pomocí mobilního telefonu.

Hlavní myšlenkou bylo nahradit již původní ruční ovládání modernějším a flexibilnějším řešením, které využívá mobilní aplikaci jako přirozené uživatelské rozhraní. Robotické rameno je řízeno mikrokontrolerem Arduino a komunikace mezi hardwarem a mobilní aplikací probíhá bezdrátově prostřednictvím technologie Bluetooth. Výsledkem je systém, který umožňuje pohodlné ovládání jednotlivých os robotického ramene bez nutnosti přímého fyzického zásahu do řídicí elektroniky.

Cílem práce je právě vytvořit funkční, přehlednou a uživatelsky dostupnou mobilní aplikaci, která umožní ovládat pohyby jednotlivých částí robotického ramene a současně nabídne možnost vytvářet a spouštět předem definované automatizované sekvence pohybů. Důležitou součástí řešení je nejen samotné bezdrátové řízení pěti os robotického ramene, ale také návrh vhodného způsobu přenosu příkazů, jejich zpracování na straně mikrokontroléru a návazné řízení servomotorů. Práce tedy nesměřuje pouze k vytvoření aplikace jako samostatného softwarového produktu, ale k vytvoření funkčního propojeného celku, v němž spolupracuje mobilní zařízení, komunikační modul i mechanický systém robotické ruky.

Z hlediska softwaru bylo třeba navrhnout mobilní aplikaci ve frameworku (předpřipravená softwarová struktura pro programování) Flutter, vytvořit vhodné uživatelské rozhraní a zajistit spolehlivou Bluetooth komunikaci. Z hlediska hardwaru bylo nutné vycházet z konstrukce robotického ramene, způsobu zapojení servomotorů do Adafruit servořadiče a mikrokontroléru Arduino UNO. Významným úkolem bylo také navrhnout takový způsob ovládání, který bude pro uživatele srozumitelný, rychlý a dostatečně přesný, a zároveň umožní nejen manuální řízení

jednotlivých os a automatizované vykonávání uložených pohybových sekvencí, ale i nastavení rychlosti otáčení a vytvoření nastavení pro uživatelské přizpůsobení.

Přínos práce také spočívá v ukázce, že i s využitím běžně dostupných prostředků lze vytvořit takovýto projekt. Projekt může najít využití nejen při výuce a demonstraci, ale také jako základ pro další rozšiřování, například v oblasti senzorů a zpětné vazby.

2 TEORETICKÁ ČÁST

2.1 Robotická ramena a jejich využití

Robotická ramena patří mezi nejznámější typy technických zařízení používaných pro manipulaci s předměty a vykonávání opakovaných pohybů. Jejich princip vychází z lidské paže, protože se skládají z několika navazujících částí spojených klouby. Díky tomu mohou vykonávat pohyb v různých směrech a dosahovat do prostoru podle konstrukčního uspořádání. Robotická ramena se využívají především v průmyslu, kde slouží například k montáži, třídění, přesunu materiálu nebo obsluze automatických linek. Kromě průmyslového nasazení se však stále objevují také ve školních výukách, při výuce technických oborů nebo v hobby konstrukcích.

Význam robotických ramen spočívá zejména v jejich schopnosti přesně a opakovatelně vykonávat zadané úlohy. Oproti člověku mají výhodu v tom, že se neunaví, mohou pracovat dlouhodobě stejným způsobem a při správném nastavení dokážou dosahovat velmi podobných výsledků při každém opakování pohybu.

Pro vzdělávací účely jsou robotická ramena vhodná proto, že spojují několik oblastí techniky do jednoho funkčního celku. Při jejich návrhu je nutné řešit mechanickou konstrukci, volbu pohonů, způsob napájení, řízení pomocí mikrokontroléru i uživatelské ovládání. Takový projekt proto poskytuje dobrou možnost propojit teoretické poznatky s praktickou realizací.

2.2 Konstrukce a řízení robotického ramene

Každé robotické rameno je tvořeno mechanickou soustavou článků a kloubů. Jednotlivé klouby umožňují pohyb v určitém směru a dohromady vytvářejí počet stupňů volnosti ramene. Stupeň volnosti vyjadřuje, kolika nezávislými způsoby se může zařízení pohybovat [1]. U jednodušších ramen se běžně používá několik rotačních os, které umožňují otáčení základny, pohyb ramene ve svislé rovině, změnu polohy dalších částí a ovládání koncového efektoru, například chapadla.

Počet stupňů volnosti přímo ovlivňuje možnosti pohybu zařízení. Čím více os robotické rameno má, tím větší je jeho pohybová flexibilita. Současně se ale zvyšuje složitost řízení, protože je třeba koordinovat více pohybů najednou. Někdy se proto často volí kompromis mezi mechanickou jednoduchostí a praktickou použitelností. Pěti-osé robotické rameno již umožňuje poměrně široké spektrum pohybů, přičemž zůstává konstrukčně i programově zvládnutelné.

Řízení robotického ramene spočívá v převodu uživatelského požadavku na konkrétní pohyb jednotlivých pohonů. V jednodušších systémech se každá osa ovládá samostatně nastavením určité polohy motoru. Ve složitějších systémech se používají výpočty kinematiky, které umožňují zadávat požadovanou polohu koncového bodu v prostoru.

Důležitým požadavkem na řízení je plynulost a bezpečnost pohybu. Náhlé změny polohy mohou zvyšovat mechanické namáhání konstrukce a snižovat přesnost. Proto je vhodné zavést omezení rozsahů pohybu, definovat výchozí polohy a řídit rychlost přechodu mezi jednotlivými stavy. Tyto principy jsou důležité jak při manuálním ovládání, tak při automatickém vykonávání sekvencí.

2.3 Mikrokontrolery v robotice se zaměřením na Arduino

Mikrokontrolér představuje základní řídicí prvek mnoha robotických a automatizačních systémů. Jde o programovatelný elektronický obvod, který přijímá vstupy, zpracovává data a na základě programu ovládá výstupy [2]. V robotice se mikrokontrolery používají pro řízení motorů, čtení senzorů, komunikaci s dalšími zařízeními a koordinaci činnosti celého systému.

Pro výukové a hobby projekty patří mezi nejčastěji používané platformy Arduino. Důvodem je zejména jeho dostupnost, jednoduché programování a široká komunita uživatelů. Arduino umožňuje snadné připojení servomotorů, komunikačních modulů i dalších periférií. Výhodou je také existence množství knihoven, které usnadňují implementaci běžných funkcí bez nutnosti řešit všechny detaily na nejnižší úrovni.

Nevýhodou jednodušších mikrokontrolerů může být omezený výkon, menší paměť a omezené možnosti složitějších výpočtů. Pro účely této práce však taková platforma plně postačuje.

2.4 Servomotory a jejich použití v robotických mechanismech

Servomotory patří mezi často používané pohony v menších robotických konstrukcích. Jejich hlavní výhodou je možnost poměrně přesně nastavit požadovanou polohu výstupní hřídele. Běžný modelářský servomotor obsahuje elektromotor, převodový mechanismus, řídicí elektroniku a zpětnovazební prvek (potenciometr). Díky tomu lze servomotoru zadat konkrétní úhel natočení a pohon se snaží této polohy dosáhnout. [3]

Při použití servomotorů je nutné počítat s jejich omezeními. Patří mezi ně zejména omezený rozsah pohybu, konečný moment a citlivost na zatížení. Z toho důvodu je při návrhu robotického ramene důležité vhodně rozložit hmotnost konstrukce, zvolit správné převody a nastavit bezpečné krajní polohy. Správná volba servomotorů i jejich řízení má přímý vliv na stabilitu a životnost celého zařízení.

2.5 Bezdrátová komunikace Bluetooth při řízení zařízení

Bluetooth je běžně používaná bezdrátová technologie určená pro komunikaci na krátkou vzdálenost. V technických projektech se využívá především tam, kde je potřeba jednoduché propojení dvou zařízení bez použití kabelu. Výhodou Bluetooth je široká podpora v mobilních telefonech a relativně snadná implementace v menších elektronických systémech.

V oblasti hobby elektroniky se často používají moduly HC-05 nebo HC-06 (u tohoto projektu HC-05). Tyto moduly pracují s tzv. Bluetooth Classic a podporují sériovou komunikaci přes profil SPP (serial port profile). Takový přístup je vhodný zejména pro Arduino, protože přenášená data se z pohledu programu chovají podobně jako běžná sériová linka. Flutter pluginy pro Bluetooth Classic na Androidu běžně podporují párovaná zařízení, RFCOMM/SPP (Radio Frequency Communication) spojení a asynchronní přenos dat, což odpovídá způsobu řízení zařízení jako HC-05 a Arduino [4]. Zároveň je důležité odlišit Bluetooth Classic od BLE (Bluetooth Low Energy), protože některé moderní Flutter knihovny podporují pouze BLE a pro HC-05 se nehodí.

Při řízení robotického zařízení přes Bluetooth je důležité zajistit spolehlivý a srozumitelný přenos příkazů. V jednoduchých systémech se často používají textové řetězce s pevně daným formátem, například hodnoty oddělené čárkami. Výhodou tohoto řešení je snadná implementace a jednoduché ladění. Na druhou stranu je třeba ošetřit chyby přenosu, duplicitní příkazy nebo příliš rychlé odesílání dat, které by mohlo zařízení zahlcovat.

Bluetooth komunikace má i určitá omezení. Dosah je relativně malý a kvalita spojení může být ovlivněna překážkami nebo rušením. U Bluetooth Classic je navíc třeba počítat s tím, že podpora na různých platformách není stejná. Některá řešení proto fungují zejména na systému Android.

2.6 Mobilní aplikace pro řízení technických systémů

Při návrhu mobilní aplikace pro řízení technického systému je důležitá především jednoduchost ovládání. Uživatel musí rychle pochopit, jak aplikace funguje, a musí mít okamžitou zpětnou vazbu o stavu systému. V případě robotického ramene je vhodné použít prvky, které umožní intuitivně nastavovat polohu jednotlivých os, například posuvníky, tlačítka nebo přednastavené akce. Důležitá je také přehledná navigace mezi jednotlivými obrazovkami a zobrazení stavu připojení.

2.7 Framework Flutter a jeho využití v projektu

Flutter je framework určený pro vývoj mobilních aplikací. Je založen na programovacím jazyce Dart a umožňuje vytvářet přehledná uživatelská rozhraní z jednoho jazyka. Jeho výhodou je rychlý vývoj, nativita, dobrá práce s grafickými prvky a možnost rozdělit aplikaci do samostatných obrazovek a logických částí.

V tomto projektu je pro bezdrátovou komunikaci využita knihovna `'flutter_bluetooth_serial'` [4], která umožňuje pracovat s Bluetooth Classic zařízeními v prostředí Androidu.

2.8 Automatizované sekvence pohybů

Vedle přímého ručního ovládání představují důležitou součást řízení robotických mechanismů také automatizované sekvence. Sekvence je předem definovaný sled kroků, které určují, jaké pohyby mají být vykonány, v jakém pořadí a s jakým časovým odstupem. Tento princip umožňuje opakované provádění stejné činnosti bez nutnosti zásahu uživatele při každém jednotlivém kroku.

Výhodou sekvenčního řízení je zejména opakovatelnost. Pokud uživatel jednou vytvoří správný sled pohybů, může jej následně spouštět opakovaně se stejným výsledkem. To je užitečné například při uchopení a přemístění předmětu nebo při demonstraci určité pracovní operace.

3 CÍLE PRÁCE

Tato práce se zabývá návrhem a realizací systému pro ovládání robotického ramene pomocí mobilního telefonu. Hlavním cílem práce je vytvořit funkční a uživatelsky přístupnou mobilní aplikaci, která umožní bezdrátové řízení pohybů jednotlivých částí pěti-osého robotického ramene a současně nabídne možnost vytvoření a spuštění vlastního automatizovaného pohybu ve formě sekvence. Práce tak reaguje na potřebu propojit mechanickou konstrukci robotického zařízení s moderním a přehledným způsobem ovládání prostřednictvím mobilní aplikace.

Počátečním cílem práce bylo navrhnout a sestavit funkční celek složený z robotického ramene, řídicí elektroniky a mobilní aplikace. Součástí řešení je výběr vhodného volně dostupného modelu robotického ramene, jeho fyzická realizace pomocí 3D tisku, sestavení mechanické konstrukce, zapojení jednotlivých elektronických komponent a vytvoření firmwaru pro mikrokontrolér. Na tento základ navazuje vývoj softwarové části, tedy mobilní aplikace pro ovládání zařízení.

Hlavním cílem práce je navrhnout a implementovat mobilní aplikaci, která umožní navázání Bluetooth spojení s robotickým ramenem a následné ovládání jeho jednotlivých os pomocí mobilního telefonu a spouštění sekvencí. Důležitou součástí je vytvoření přehledného uživatelského rozhraní, které bude snadno ovladatelné a umožní jak manuální řízení, tak práci s automatizovanými sekvencemi pohybů.

4 VLASTNÍ PROJEKT

4.1 Konstrukční řešení

Výchozím bodem maturitní práce je malé stolní robotické rameno, jehož základní konstrukční model byl převzat z dostupného hobby projektu na internetu [5] [6]. Tento model byl následně upraven a rozšířen podle cílů práce, především v oblasti řízení, elektroniky a způsobu ovládání.

Hlavní řídicí část robota tvoří Arduino Uno¹, které je doplněno o externí PWM (pulzně šířková modulace) servo řadič od Adafruit², jenž zajišťuje generování signálů pro jednotlivé servomotory. Toto řešení umožňuje spolehlivé a přesné ovládání více serv současně a odlehčuje hlavnímu mikrokontroléru.

4.1.1 Důvod výběru původního modelu robotického ramene

Původní model robotického ramene byl vybrán z několika důvodů. Má jednoduchou a přehlednou mechanickou konstrukci, obsahuje pět stupňů volnosti, je navržen pro použití běžně dostupných servomotorů, konstrukce je kompaktní, modulární a umožňuje další úpravy a většina částí modelu je snadno vytisknutelná.

Důležitým faktorem byla také skutečnost, že původní model byl veřejně dostupný a dobře autorem zdokumentovaný, což umožnilo jeho pochopení, sestavení a následnou úpravu bez nutnosti návrhu celého robota od začátku.

4.1.2 Původní způsob ovládání a jeho omezení

V původním provedení bylo robotické rameno ovládáno pomocí samostatně sestaveného fyzického ovladače z potenciometrů, který byl připojen k řídicí jednotce kabelově. Ovládání probíhalo manuálně, bez možnosti bezdrátové komunikace nebo rozšíření o pokročilejší způsoby řízení, a proto jsem se rozhodl udělat vlastní.

4.1.3 Změna konstrukční koncepce – ovládání pomocí Bluetooth

Jednou z hlavních úprav oproti původnímu návrhu je změna způsobu ovládání robota. Místo původního fyzického ovladače je robot ovládán bezdrátově pomocí Bluetooth modulu

¹ Arduino UNO, teorie: https://cs.wikipedia.org/wiki/Arduino_Uno

² Adafruit PCA9685, <https://learn.adafruit.com/16-channel-pwm-servo-driver/overview>

HC-05³ a mobilní aplikace. Bluetooth modul je připojen k Arduino a umožňuje bezdrátový přenos řídicích povelů, pohodlné ovládání robota na dálku, snadné rozšíření o další funkce v aplikaci, sekvence.

Tato úprava výrazně zvyšuje uživatelský komfort a současně rozšiřuje možnosti dalšího vývoje projektu, například o automatizované sekvence pohybů.

4.1.4 Mechanická konstrukce a stupně volnosti

Robotické rameno má celkem **pět stupňů volnosti**, které odpovídají následujícím částem:

- základna (base),
- rameno (shoulder),
- loket (elbow),
- zápěstí (wrist),
- ruka (hand).

Každý kloub je ovládán samostatným servomotorem. Pro většinu kloubů jsou použita standardní MG serva, zatímco v kloubu shoulder je použit výkonnější servomotor, který musí nést největší mechanické zatížení.

Konkrétně jsou to tyto serva:

- (3x) serva MG996R (base, elbow, wrist) – 180°
- sg90 micro servo (hand) – 180°
- HPS-2018MG (shoulder) – 180°

³ HC – 05 je zařízení umožňující připojení k bluetooth, více o něm na: <https://www.sakul.cz/bluetooth-hc-05/n/>



Obrázek 1. Popis jednotlivých stupňů volností, vytvořeno ve Fusion360, Screenshot. Autor

4.1.5 Konstrukční řešení zápěstí a tvorba

Zápěstí robota je řešeno pomocí soustavy tří ozubených kol, která jsou napojena na servomotor, umožňující přenos otáčivého pohybu na osu zápěstí. Koncový efektor – ruka (hand) – je ovládán mikroservem, které postačuje díky nízkému zatížení této části robota. Robotické rameno bylo vytvořeno v CAD programu Fusion360. Všechny části robota byly vytištěny ve škole na 3D tiskárně z PLA filamentu, kromě ozubených koleček a šroubů.

4.1.6 Elektronika a napájení

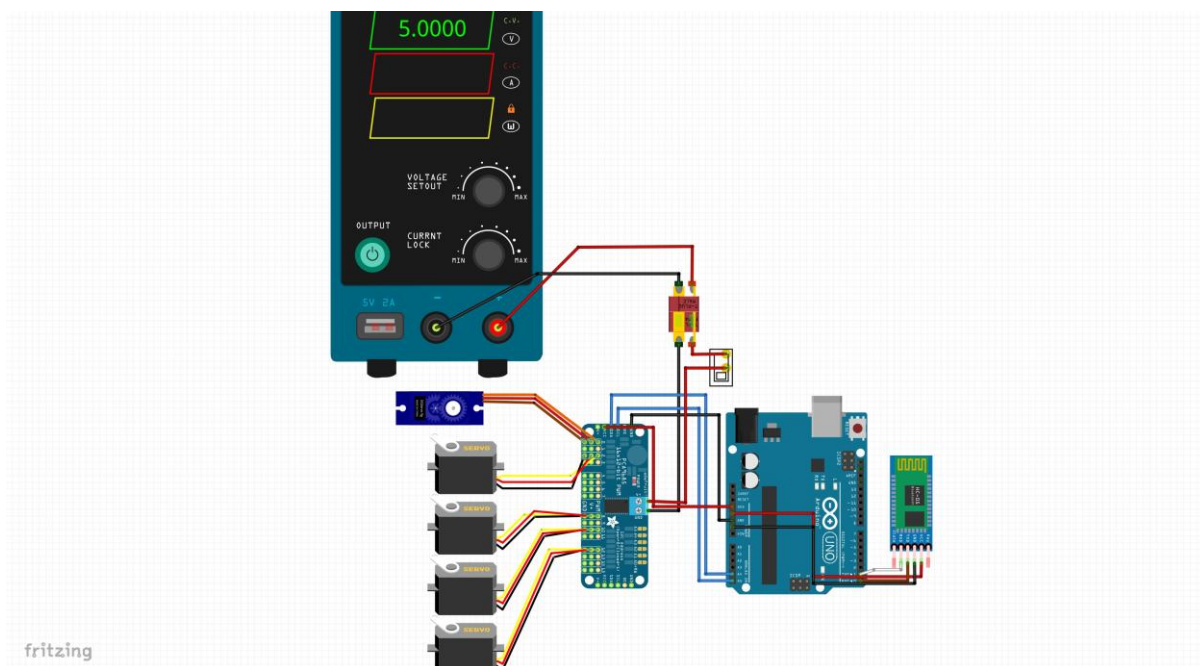
Ve spodní části robota, pod základnou, je umístěno Arduino Uno, Adafruit PWM (Pulse Width Modulation) Servo Driver (PCA9685) a Bluetooth modul HC-05.

Robot je napájen proměnlivým externím zdrojem nastaveným na 5 V, který je přiveden přes spínač do servo řadiče. Servo řadič napájí přímo všechna serva. Arduino je napájeno samostatně přes USB-B konektor, z něhož je dále napájen Bluetooth modul a logická část servo řadiče.

Prakticky je zde využít spínač, který musí být vždy vypnutý předtím, než připojíme napájení. Jakmile nastavíme správně 5 V a připojíme plus a zem, překontrolujeme znovu

hodnotu a poté zapneme spínač. Pro lepší bezpečnost není toto napájení využito přímo na čip PCA9865 a mikrokontrolery ostatních periférií, ale radši je využito USB – B, které napájí Arduino, bluetooth modul a logiku PCA9865.

Tento způsob zapojení napájení zvyšuje stabilitu a bezpečnost systému a zabraňuje výpadkům při zatížení serv.



Obrázek 2. Schéma zapojení veškeré elektroniky, vytvořeno ve fritzing. Schéma Autor

4.2 Vývojové nástroje pro tvorbu mobilní aplikace

Pro realizaci mobilní aplikace určené k ovládání robotického ramene bylo nutné zvolit vhodné vývojové nástroje jak pro vývoj aplikace, tak pro programování řídicí elektroniky robota. Při výběru nástrojů byl kladen důraz na dostupnost, podporu platformy Android, možnost ladění a rozšiřitelnost projektu.

4.2.1 Zdůvodnění volby frameworku Flutter a jazyka Dart

Framework Flutter⁴ s programovacím jazykem **Dart**⁵ byl zvolen na základě výhod, mezi které patří plynulé uživatelské rozhraní, spolehlivá Bluetooth komunikace a řízení robotického ramene v reálném čase. Aplikace musí rychle reagovat na uživatelské vstupy, odesílat řídicí

⁴ Flutter (Framework), více o něm na: [https://cs.wikipedia.org/wiki/Flutter_\(software\)](https://cs.wikipedia.org/wiki/Flutter_(software))

⁵ Programovací jazyk který používá Flutter, více o něm na [https://cs.wikipedia.org/wiki/Dart_\(programovac%C3%AD_jazyk\)](https://cs.wikipedia.org/wiki/Dart_(programovac%C3%AD_jazyk))

pověly a současně zůstat stabilní. Flutter umožňuje vytvářet aplikace, které jsou kompilovány do nativního kódu a nevyužívají webovou vrstvu ani dodatečné běhové prostředí, což přispívá k předvídatelnému výkonu a nízké odezvě aplikace.

Hlavní výhodou Flutteru v tomto projektu není to, že by jiné programovací jazyky nebo technologie nebyly schopné podobnou aplikaci vytvořit, ale to, že Flutter umožňuje řešit uživatelské rozhraní, aplikační logiku i Bluetooth komunikaci v jednom prostředí a jednom jazyce. Díky tomu je struktura aplikace jednodušší, přehlednější a lépe se udržuje. Tyto vlastnosti jsou v projektu využity například při přímém propojení ovládacích prvků s řízením servomotorů a při implementaci pohybových sekvencí robota.

4.2.2 Vývojové prostředí pro mobilní aplikaci

Pro vývoj mobilní aplikace bylo zvoleno prostředí Visual Studio Code⁶, které slouží jako jednoduché, ale výkonné vývojové prostředí. VS Code bylo doplněno o rozšíření Flutter-framework pro vývoj multiplatformních aplikací, Dart – programovací jazyk používaný frameworkem Flutter, GitHub Copilot – AI nástroj pro asistenci při psaní kódu. GitHub Copilot mi výrazně pomohl zejména při implementaci složitějších funkcí, správné konfiguraci Gradle souborů, nastavení SDK (software development kit) verzí a řešení kompatibility různých knihoven. Pomohl také s laděním konfiguračních problémů.

4.2.3 Android Studio a vývojové závislosti

Aby bylo možné aplikaci spouštět a testovat na zařízení s operačním systémem Android, bylo nutné nainstalovat Android Studio⁷, které poskytuje Android SDK⁸, nástroje pro emulaci zařízení a ovladače pro komunikaci s fyzickým Android telefonem. V Android Studiu byla nastavena správná verze Android SDK a propojena s prostředím Flutter. Bez této konfigurace by nebylo možné aplikaci sestavit, spustit ani ladit.

4.2.4 Konfigurace vývojového prostředí

Součástí přípravy vývojového prostředí bylo také provedení řady konfiguračních kroků v příkazovém řádku PowerShell. Tyto kroky zahrnovaly zejména instalaci frameworku Flutter,

⁶ Visual studio code je editor zdrojového kódu, více o něm na: https://cs.wikipedia.org/wiki/Visual_Studio_Code

⁷ Android studio je vývojové prostředí pro Android aplikace, více na: https://cs.wikipedia.org/wiki/Android_Studio

⁸ Android SDK je Android softwarový vývojářský balíček, více na: https://en.wikipedia.org/wiki/Android_SDK

nastavení systémových proměnných, ověření funkčnosti nástrojů pomocí příkazu flutter doctor, propojení Flutteru s Android SDK. Tento proces byl nezbytný pro správnou funkci vývojového prostředí a umožnil ladění aplikace přímo na připojeném Android zařízení.

4.2.5 Debugování a testování aplikace

Po dokončení konfigurace bylo možné aplikaci spouštět na fyzickém Android mobilu, ladit v reálném čase (Debugovat) a testovat Bluetooth komunikaci s robotem. Debugování přímo na zařízení umožnilo ověřit správnou funkci uživatelského rozhraní i komunikace s Bluetooth modulem HC-05.

4.2.6 Vývojové nástroje pro programování robota

Pro programování řídicí jednotky robota bylo použito Arduino IDE (integrované vývojové prostředí), které slouží k psaní a nahrávání programu do Arduino UNO, testování komunikace přes sériovou linku a ladění chování servomotorů.

Ukázková funkce zpracování příkazu (processCommand) z firmwaru pro Arduino UNO (jazyk c++).

```
// Očekává "servoChannel,angle,speed"
void processCommand(String cmd) {
    int firstComma = cmd.indexOf(',');
    int secondComma = cmd.indexOf(',', firstComma + 1);

    if (firstComma == -1 || secondComma == -1) {
        Serial.println("Invalid command format.");
        return;
    }

    int servoChannel = cmd.substring(0, firstComma).toInt();
    int angle = cmd.substring(firstComma + 1, secondComma).toInt();
    int speed = cmd.substring(secondComma + 1).toInt();

    // Use non-blocking target setting
    setServoTarget(servoChannel, angle, speed);
}
```

Tato funkce z přijatého textového příkazu ve formátu servoChannel,angle,speed oddělí číslo serva, požadovaný úhel a rychlost pohybu, zkontroluje správnost formátu a následně předá tyto hodnoty funkci pro nastavení cílové polohy servomotoru.

Protože Bluetooth v režimu SPP přenáší data jako sériový proud znaků, je nutné jednoznačně určit strukturu přenášených příkazů. V opačném případě může dojít k chybné interpretaci dat na straně mikrokontroléru. V praxi to znamená, že příkazy musí mít definovaný začátek a konec, jednotlivé hodnoty musí být odděleny známým znakem (např. čárkou nebo středníkem), mikrokontrolér musí přijatá data správně parsovat⁹. Správně navržený formát přenášených dat je zásadní pro stabilní řízení servomotorů a předcházení nechtěným pohybům robota.

4.3 Návrh interface obslužné aplikace

4.3.1 Účel aplikace a cílový uživatel

Obslužná aplikace slouží jako uživatelské rozhraní pro řízení robotického ramene bezdrátovou komunikací. Cílovým uživatelem je běžný operátor, který potřebuje vyhledat a připojit Bluetooth modul, ručně nastavovat polohy serv, spouštět předpřipravené sekvence pohybů. Z pohledu návrhu je důležité, aby rozhraní bylo přehledné, mělo jasné „bezpečné“ ovládací prvky a minimalizovalo riziko nechtěného pohybu robota. Proto jsou v uživatelském rozhraní také implementované bezpečnostní prvky (zámky). Dále je dobré vzít do úvahy zlepšení uživatelského komfortu například implementací obrazovky s nastavením, kde můžeme měnit tmavý/světlý motiv.

4.3.2 Použité technologie a principy UI (User interface)

Aplikace využívá knihovnu **GetX**, která je použita pro **navigaci mezi obrazovkami**, **správu stavu** a reaktivní aktualizace UI (Obx) a pro jednoduché dialogy / přechody.

Tuto knihovnu definujeme tímto způsobem:

```
import 'package:get/get.dart';
```

Základní struktura aplikace je definována v souboru main.dart.

⁹ Parsovat – (Parsing neboli Syntaktická analýza), teorie:
https://cs.wikipedia.org/wiki/Syntaktick%C3%A1_anal%C3%BDza

Getx a flutter bluetooth serial nejsou ale jediné knihovny využité v aplikaci. material.dart [7], services.dart [8], get_storage.dart [9], dart:convert [10], dart:typed_data [11], permission_handler.dart [12], dart:async [13], file_picker.dart [14] a file_bytes_reader.dart [15] patří mezi další knihovny.

Tato část kódu definuje jednotlivé obrazovky, například /servo-control je následně zavolána po připojení k bluetooth.

```
return Obx(() => GetMaterialApp( //GetMaterialApp je rozšíření MaterialApp z GetX balíčku, které poskytuje funkce pro spr
debugShowCheckedModeBanner: false, //vypnutí debug banneru v pravém horním rohu aplikace
title: "Bluetooth Servo Control", //název aplikace
theme: ThemeData( //světlé téma aplikace
  colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo), //barevná schéma založené na semínkové barvě indigo
  useMaterial3: true, //použití Material Design 3 stylu
), // ThemeData
darkTheme: ThemeData( //tmavé téma aplikace
  colorScheme: ColorScheme.fromSeed(
    seedColor: Colors.indigo,
    brightness: Brightness.dark,
  ), // ColorScheme.fromSeed
  useMaterial3: true,
), // ThemeData
themeMode: settingsController.themeMode.value, //nastavení režimu tématu ze SettingsController
getPages: [ //seznam stránek pro navigaci v aplikaci
  GetPage(name: '/', page: () => const HomeScreen()), //domovská stránka - stránka pro skenování Bluetooth zařízení
  GetPage(name: '/servo-control', page: () => const ServoControlScreen()), //stránka pro ovládání serva
  GetPage(name: '/sequence', page: () => const SequenceScreen()), //stránka pro sekvenční ovládání serva
  GetPage(name: '/settings', page: () => const SettingsScreen()), //stránka pro nastavení aplikace
],
initialRoute: '/', //počáteční trasa aplikace je domovská stránka
)); // GetMaterialApp // Obx
```

Rozhraní je rozdělené do 4 hlavních obrazovek:

1. Domovská obrazovka – Bluetooth sken (Home Screen)

- výchozí obrazovka aplikace ('/),
- slouží pro vyhledání dostupných Bluetooth zařízení a připojení.

2. Ovládání serv/robota (Servo Control Screen)

- manuální ovládání poloh, rychlosti a odemčení

3. Sekvence pohybu (Sequence Screen)

- načtení / správa sekvenčních kroků,
- určené pro opakovatelné pohyby robota.

4. Nastavení (Settings Screen)

- volba tématu (světlé/tmavé/systémové),
- uložení výchozích hodnot, informace o výchozích pinech

4.3.3 Navigace a ovládací prvky

Po spuštění aplikace se inicializují persistentní data a controllery (Bluetooth + Settings) a následně se spustí UI:

```
Run | Debug | Profile
11 void main() async { //hlavní vstupní bod aplikace, void main() = hlavní funkce bez návratové hodnoty, async = asynchronní funkce
12   WidgetsFlutterBinding.ensureInitialized(); //inicializace Flutter widgetů před spuštěním aplikace
13   await GetStorage.init(); //inicializace GetStorage pro lokální úložiště
14   Get.put(BluetoothController()); //dependency injection pro BluetoothController, dependency injection = vkládání závislostí
15   Get.put(SettingsController()); //dependency injection pro SettingsController
16   runApp(const MyApp()); //spuštění aplikace s MyApp jako kořenovým widgetem
17 }
```

Tato funkce je základní vstupní bod Flutter aplikace. Je automaticky generována při vytvoření nového projektu a slouží k inicializaci běhového prostředí a spuštění kořenového widgetu aplikace.

Na hlavní obrazovce je v AppB¹⁰aru tlačítko ukončení a tlačítko nastavení. Tlačítko nastavení přesune uživatele do obrazovky s nastavením, zatímco tlačítko ukončení ukončí aplikaci (tlačítko ukončení aplikace je i na obrazovce Ovládání serv, ale má pouze funkci vrácení na homescreen – bluetooth výběr, kde je právě to stejné tlačítko, které zase aplikaci ukončí.). Dále je v AppBaru tlačítko restartu serv do výchozích pozic, a to pouze na obrazovce Ovládání serv a Sekvence pohybu.

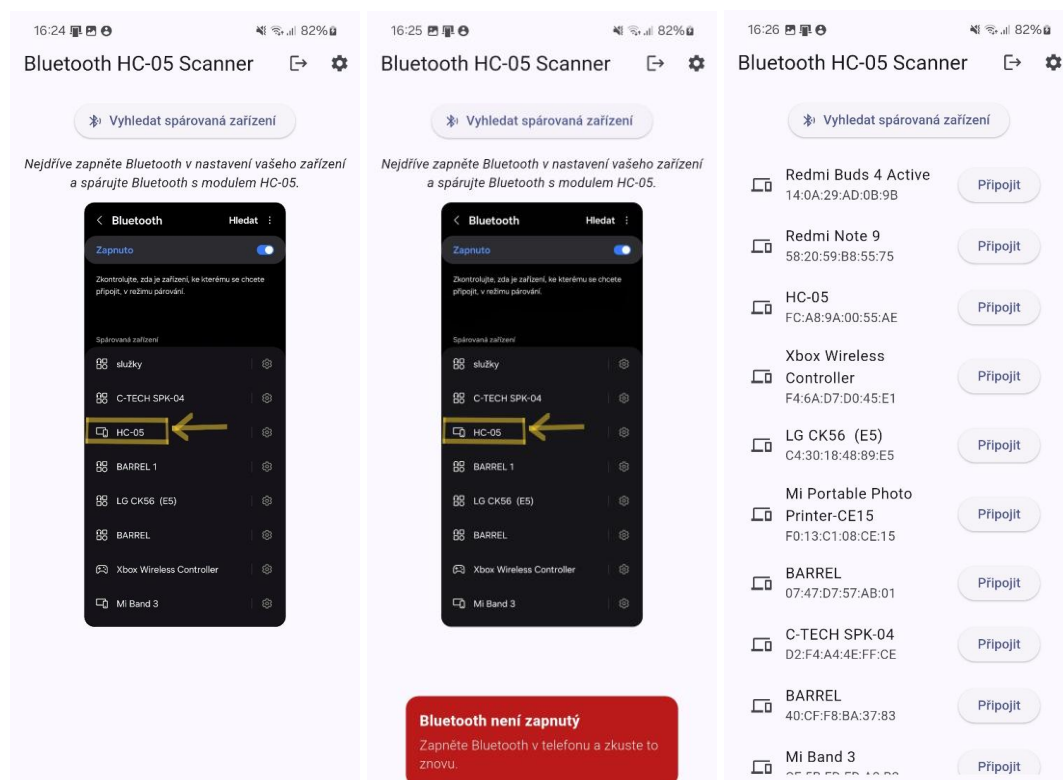
4.3.4 Domovská obrazovka – sken Bluetooth zařízení

Domovská obrazovka je navržena jako jednoduchý seznam nalezených zařízení. Pro zobrazení seznamu je použito reaktivní UI (Obx¹¹) tlačítko (Vyhledat spárovaná zařízení), které aktualizuje seznam při změně dat v controlleru. Ale ještě před tím, než se spustí, je zobrazen text a obrázek s návodem pro spárování zařízení v android nastavení mobilního zařízení. Pokud nejsme připojeni k bluetooth vyjede nám tzv. snackbar (reaktivní dialogové okno), které nás upozorní.

¹⁰ Připnutá horní lišta

¹¹ Reaktivní widget (stavební prvek) pro aktualizaci UI z knihovny GetX

Úvodní obrazovka s připojením s návodem k spárování zařízení a následná obrazovka se zařízeními po stisknutí tlačítka Vyhledat spárovaná zařízení.



Obrázek 3. Vlevo první obrazovka po načtení aplikace, uprostřed po stisknutí tlačítka Vyhledat... snackbar oznámení že bluetooth není zapnutý. Vpravo úspěšné vyhledání zařízení a seznam s připojením. Screenshoty Autor

4.4 Integrace Bluetooth

Ukázkové funkce připojení se k bluetooth, odpojení, a posílání příkazu.

```
// Připojení k zařízení
connection = await BluetoothConnection.toAddress(device.address);
isConnected.value = true;
Get.toNamed('/servo-control'); // automatická navigace po úspěchu

// Odpojení
void disconnect() {
  connection?.dispose();
  isConnected.value = false;
}

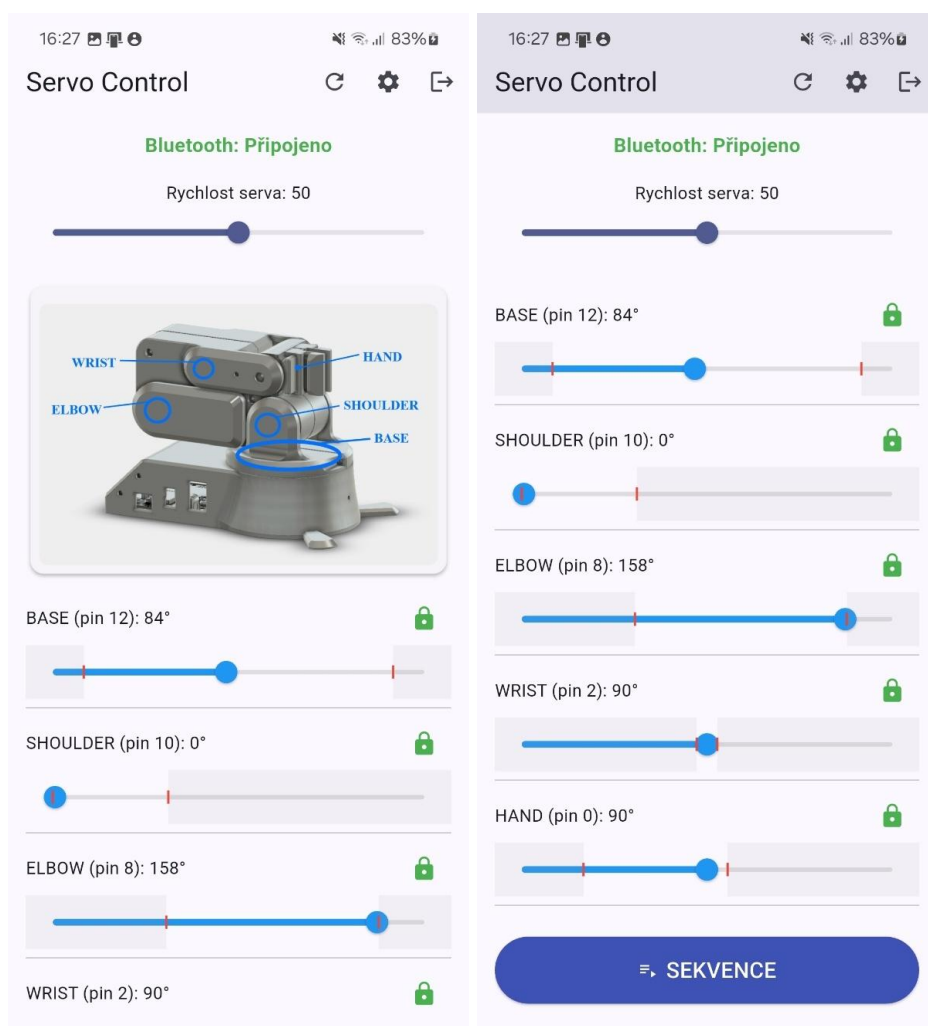
void sendServoCommand(int pin, int angle, int speed) {
  String command = '$pin,$angle,$speed\n'; // formát: "12,90,128\n"
  if (command == _lastSentCommand) return; // přeskoč duplikát
  connection!.output.add(Uint8List.fromList(utf8.encode(command)));
}
```

Ukázka kódu zachycuje základní logiku Bluetooth komunikace v aplikaci. Asynchronní funkce pro připojení pomocí `await BluetoothConnection.toAddress(...)` naváže spojení se zvoleným zařízením, po úspěchu nastaví stav připojení a automaticky otevře obrazovku pro ovládání serv. Funkce `disconnect()` naopak spojení ukončí a nastaví aplikaci do odpojeného stavu. Nejdůležitější je metoda `sendServoCommand()`, která vytvoří textový příkaz ve formátu `pin,angle,speed`, zkontroluje, zda nejde o stejný příkaz jako před chvílí, a následně jej odešle přes aktivní Bluetooth spojení do mikrokontroléru.

4.5 Obrazovka s manuálním ovládáním

Obrazovka používá Scaffold¹² + AppBar, přičemž je záměrně vypnutá šipka „zpět“. Běžné tlačítko „zpět“ by mohlo uživatele vrátit na BT obrazovku bez korektního odpojení. Místo toho se používá explicitní akce „Odpojit“, která zároveň provede správný návrat. AppBar na této obrazovce taky obsahuje již zmíněné tlačítko restartu serv do výchozích pozic a nastavení. Hned pod horní lištou je výrazný textový indikátor stavu text „Bluetooth: Připojeno“ nebo „Bluetooth: Nepřipojeno“, s barevným odlišením stavu zelená pro připojeno, červená pro nepřipojeno, tučné písmo (bold), velikost 16.

¹² Kontejner z materiál design který definuje strukturu vizuální stránky aplikace



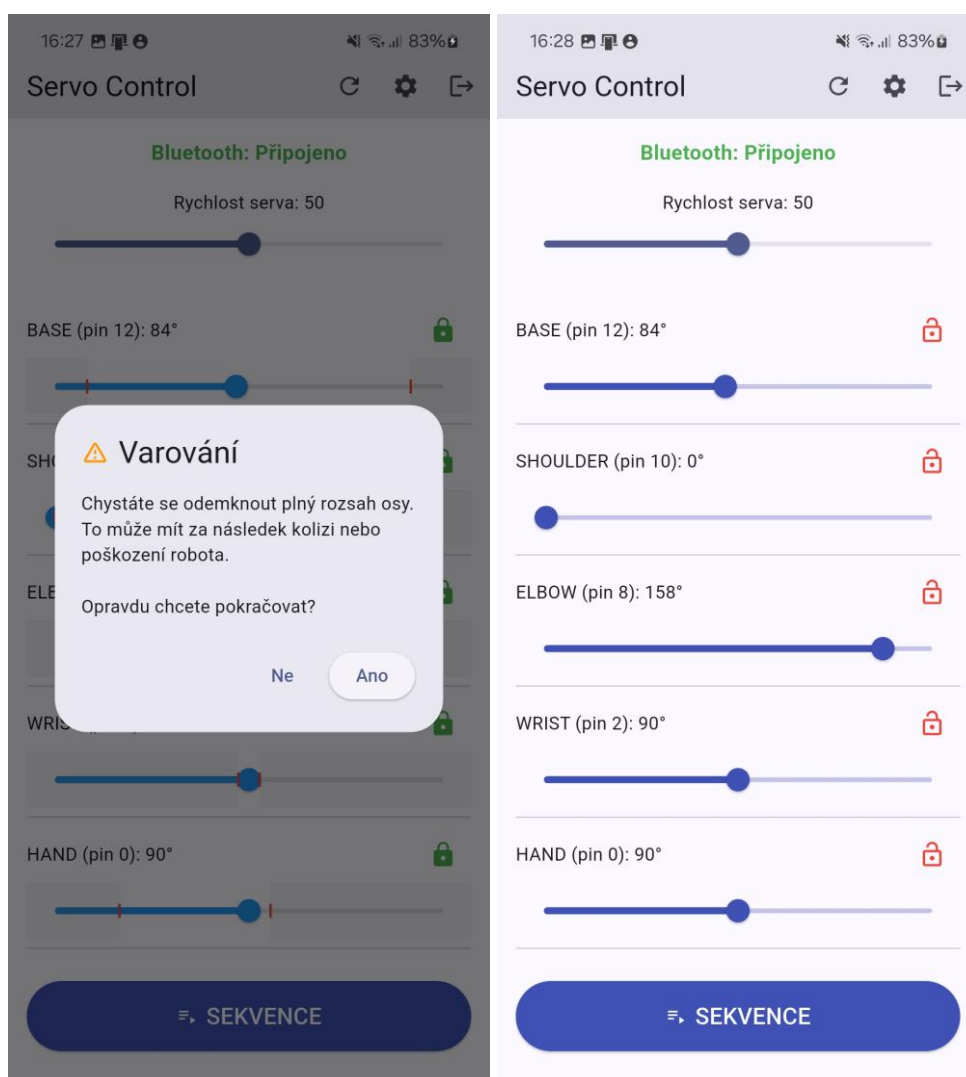
Obrázek 4. Jedna stejná scrollovatelná obrazovka-manuální ovládání serv. Vlevo jde vidět na obrazovce obrázek. Vpravo jdou po scrollnutí vidět všechny slidy. Screenshoty Autor

Pod stavem Bluetooth je samostatná sekce pro rychlost: textový popis „Rychlost serva: X“ (kde X je aktuální číslo, platí u všech sliderů), pod tím slider v rozsahu 0 až 100 po krocích 1 (divisions 100). Do arduina se převádí z hodnot 1–100 na 1–255. Z pohledu UI je to „globální“ ovládací prvek, protože je umístěný před ovládáním jednotlivých os. To znamená, že tento slider zůstane viditelný, a bude připnutý pod AppBarem, když sjedeme po obrazovce dolů. Následně lze vidět obrázek s popisem jednotlivých os pro lepší orientaci. Následně máme slidy pro jednotlivé osy s tím, že každý slider má svůj bezpečnostní zámek.

4.5.1 Bezpečnostní zámky ovládání

Součástí aplikace jsou bezpečnostní zámky, které chrání robotické rameno před nevhodným pohybem a možnou mechanickou kolizí. U jednotlivých os lze ponechat zamčený režim, ve kterém je pohyb slideru omezen pouze na bezpečný rozsah. Po odemčení je zobrazena

varovná dialogová zpráva a po potvrzení je možné využít širší rozsah pohybu, avšak není doporučeno překračovat pevná omezení vycházející z konstrukce ramene. Některé limity jsou navíc dynamické, protože se mění podle aktuální polohy jiné osy, například maximální povolený úhel ramene SHOULDER závisí na poloze ELBOW. Obdobně je při vysoké poloze ELBOW omezen také pohyb WRIST kolem výchozí polohy. Aplikace tak průběžně kontroluje, zda zadaná hodnota nepřekračuje bezpečný rozsah, a v případě potřeby ji automaticky upraví. Další ochranu představuje blokace ručního ovládání během spuštěné sekvence a omezení příliš častého odesílání příkazů, aby nedocházelo ke konfliktům v řízení a zahlcení Bluetooth komunikace.



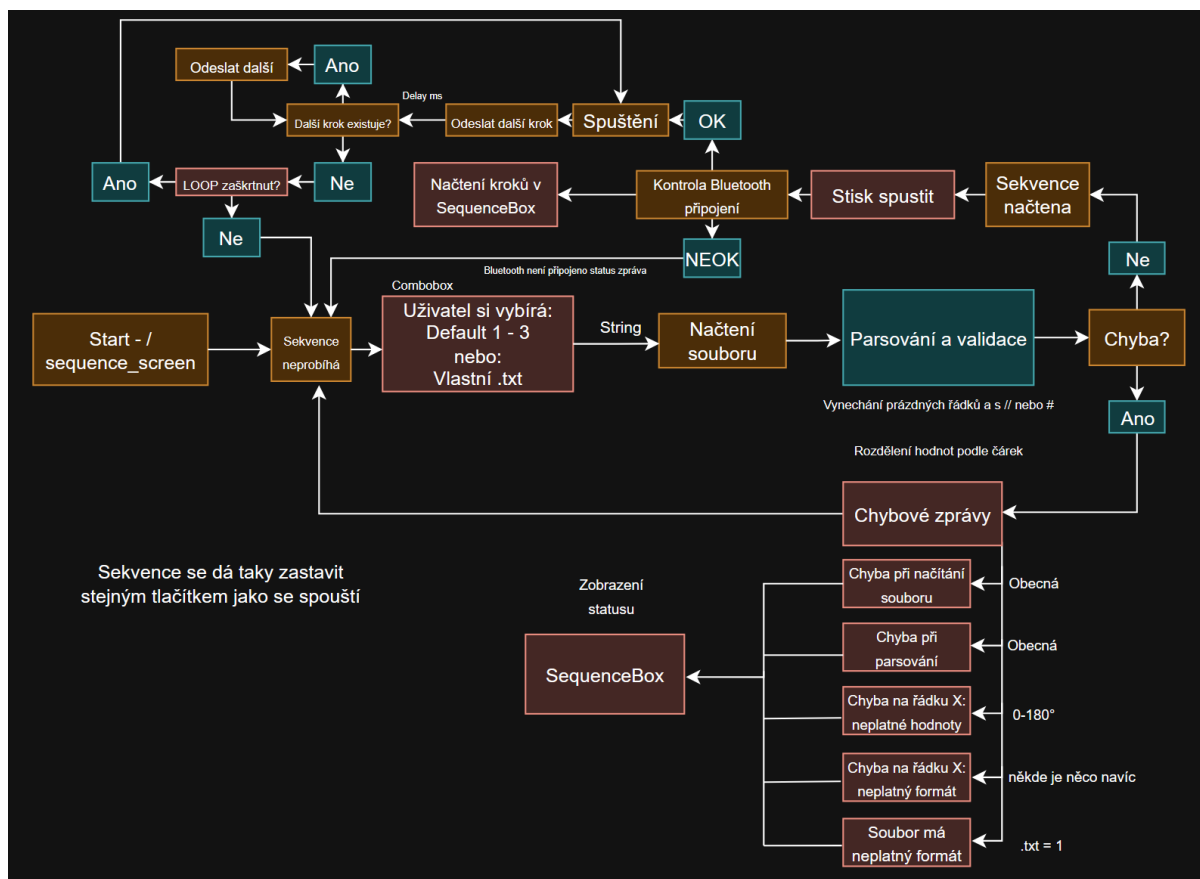
Obrázek 5. Vlevo varovné dialogové okno po stisknutí zámku. Vpravo jednotlivé odemčené slidery bez omezujících hranic. Screenshoty Autor

Pokud klikneme na tlačítko SEKVENCE, přesuneme se do sekce Sekvence.

4.6 Sekvence

Součástí aplikace je část se sekvencemi, která umožňuje automatické vykonávání předem připravených pohybů robotického ramene. Uživatel může zvolit některou z výchozích sekvencí nebo načíst vlastní textový soubor ve formátu `.txt`. Po načtení je obsah souboru zkontrolován, jednotlivé řádky jsou převedeny na kroky sekvence a následně může být sekvence spuštěna. Každý krok sekvence obsahuje číslo servokanálu, cílový úhel, rychlost pohybu a případně také zpoždění před dalším krokem. Aplikace při spuštění sekvence postupně prochází jednotlivé kroky, odesílá odpovídající příkazy přes Bluetooth a po nastaveném čase pokračuje na další krok. Díky tomu je možné provádět opakované a přesně definované pohyby bez nutnosti ručního ovládání každé osy zvlášť.

Důležitou součástí této obrazovky je také kontrola správnosti vstupních dat. Při načítání sekvence se ověřuje formát řádků, počet hodnot, číselné rozsahy i platnost jednotlivých parametrů. Pokud soubor obsahuje chybu, aplikace sekvenci nespustí a uživateli zobrazí odpovídající chybovou zprávu. Během vykonávání sekvence jsou navíc zablokovány ruční ovládací prvky ostatních částí aplikace, aby nedošlo ke konfliktu mezi ručním a automatickým řízením. Sekvence lze spouštět také v opakovacím režimu. Po dokončení všech kroků se v tomto případě robot nejprve vrátí do výchozích poloh a následně se sekvence znovu spustí od začátku. Tato funkce je vhodná například pro opakovanou demonstraci pohybu nebo dlouhodobější testování.



Obrázek 6. Diagram algoritmu sekvence. Diagram Autor

4.6.1 Formát souboru sekvence

Screenshot z textového souboru popisuje správná pravidla pro vytvoření sekvence

```

### Formát souboru sekvence

Sekvence je uložena v textovém souboru '.txt', kde každý řádek představuje jeden krok pohybu. Podporovány jsou dva základní formáty:

- `pin,angle,speed,delayMs`
- `pin,angle,speed`

Pokud není uvedeno `delayMs`, použije se výchozí zpoždění 300 ms.

#### Význam jednotlivých hodnot
- `**pin**` - číslo servokanálu
- `**angle**` - cílový úhel ve stupních
- `**speed**` - rychlost pohybu serva
- `**delayMs**` - zpoždění v milisekundách před vykonáním dalšího kroku

#### Přiřazení kanálů v projektu
- `12` = BASE
- `10` = SHOULDER
- `8` = ELBOW
- `2` = WRIST
- `0` = HAND

#### Povolené rozsahy
- `angle`: 0 až 180
- `speed`: 1 až 255
- `delayMs`: 0 a více

#### Komentáře
- řádky začínající znakem `#` jsou ignorovány
- řádky začínající `//` jsou také ignorovány
- prázdné řádky jsou přeskoceny

```

4.6.2 Ukázka kódu sekvence

```
final lines = content.split('\n');
final steps = <SequenceStep>[];

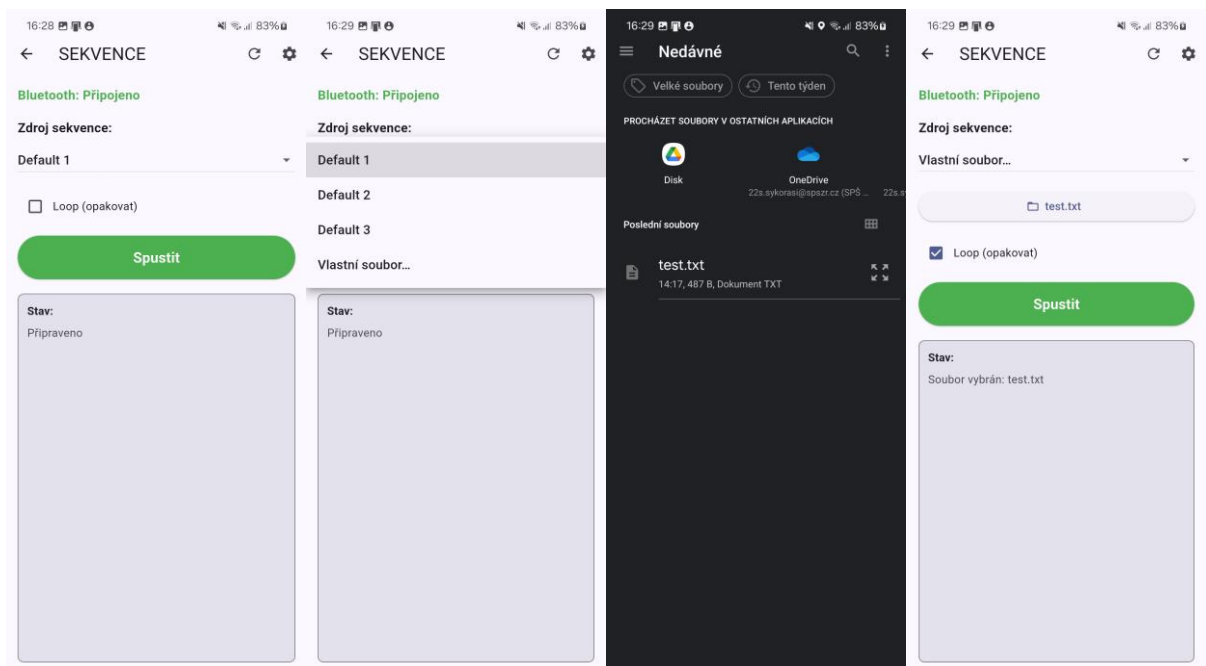
for (int i = 0; i < lines.length; i++) {
    final line = lines[i].trim();

    final parts = line.split(',');
    if (parts.length < 3 || parts.length > 4) {
        setState(() {
            // nastaví chybovou zprávu
        });
        return false;
    }
}
```

Ukázka kódu zachycuje hlavní princip parsování sekvence. Textový obsah souboru je nejprve rozdělen na jednotlivé řádky a následně je každý řádek zpracován samostatně. U každého kroku se kontroluje počet hodnot oddělených čárkou, jejich číselná správnost a případně i další omezení. Pokud je nalezena chyba, aplikace sekvenci nenačte a uživateli zobrazí odpovídající hlášení. V případě úspěchu jsou jednotlivé kroky uloženy do seznamu a aplikace oznámí, kolik kroků bylo načteno.

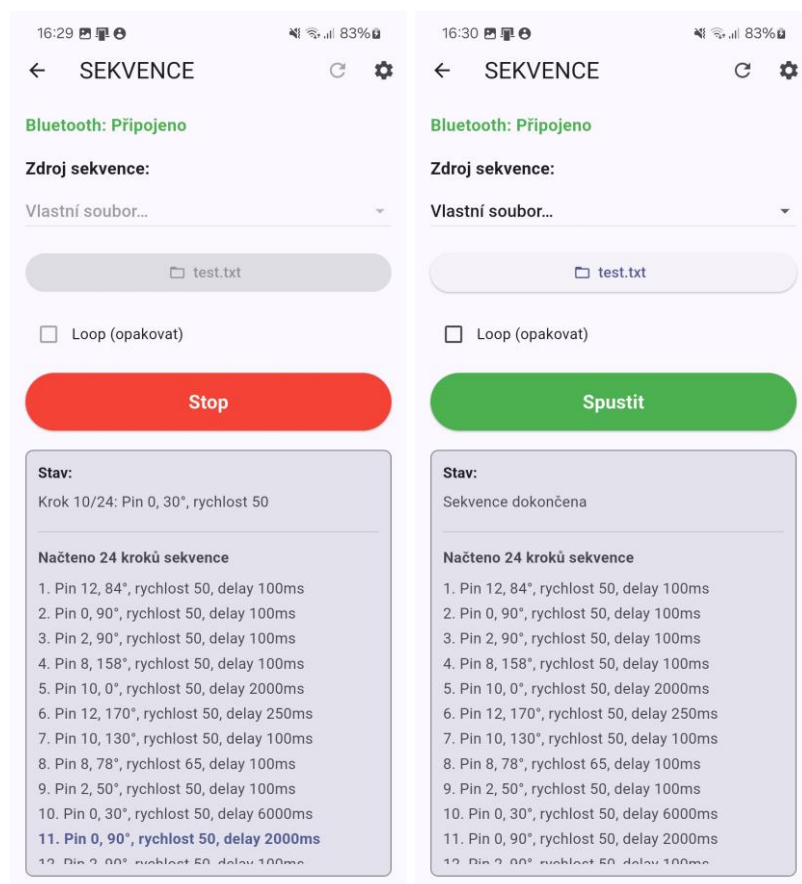
4.6.3 UI sekvence

Následující obrázky jdou za sebou jako jednotlivé kroky, zleva doprava.



Obrázek 7. První zleva je obrazovka po spuštění. Druhý zleva je rozkliknutý combobox s výběrem ukázkových defaultních pohybů a výběrem vlastního souboru. Třetí zleva je výběr souboru ze systémové aplikace po kliknutí

na Vlastní soubor... z druhého Screenshotu zleva. Čtvrtý zleva ukazuje že soubor byl úspěšně vybrán a je i zaškrtnutý loop. Screenshoty Autor

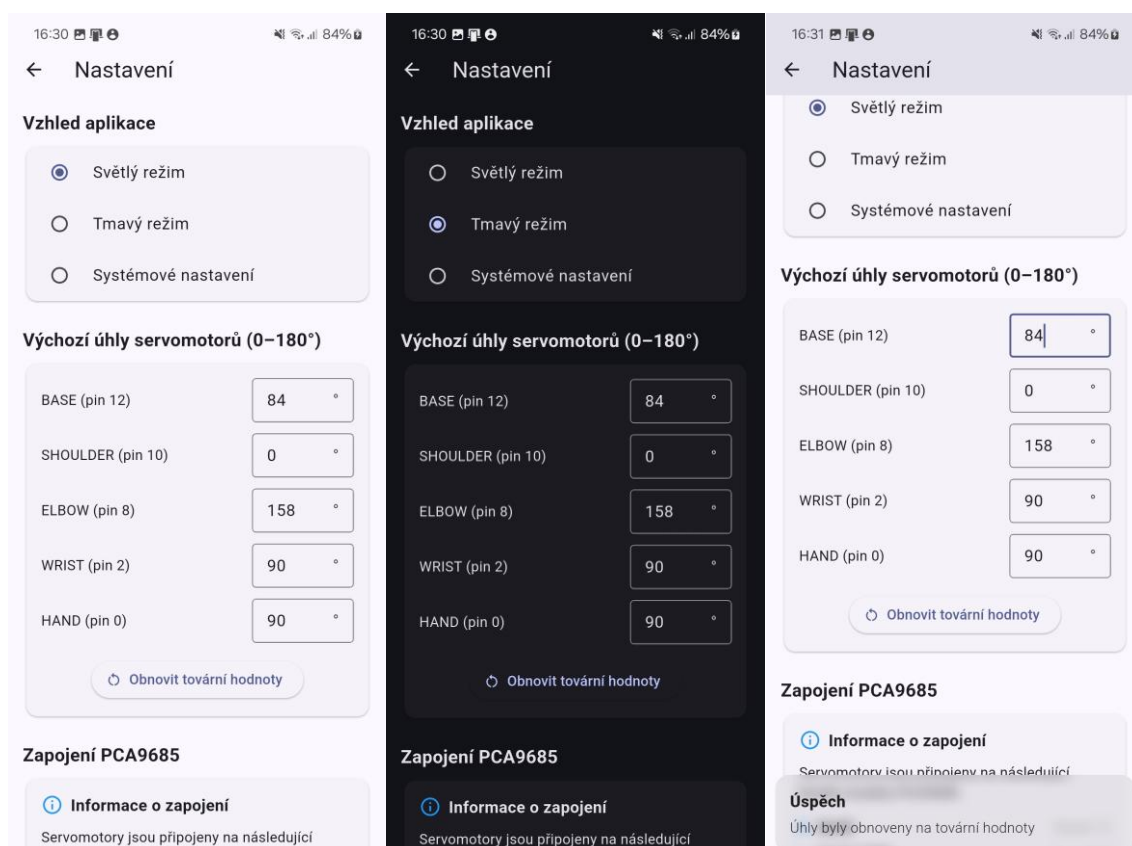


Obrázek 8. Vlevo je spuštěná sekvence po načtení a jdou vidět jednotlivé kroky. Vpravo když se sekvence zastaví nebo dokončí, Screenshoty Autor

Pokud úspěšně nahrajeme sekvenci zobrazí se v SequenceBox všechny kroky a postup.

4.7 Obrazovka s nastavením

Součástí aplikace je také obrazovka s nastavením, která slouží k úpravě výchozích úhlů jednotlivých servomotorů robotického ramene. Dostaneme se do ní přes tlačítko nastavení v AppBaru. Uživatel zde může pro každou osu zadat vlastní hodnotu pro výchozí pozice servomotorů v rozsahu 0° až 180°, které se následně uloží a využívá je resetu tlačítko ramene do výchozí polohy. Dále je pod touto nabídkou tlačítko, které vrátí hodnoty do továrního nastavení. V nastavení je taky možnost přizpůsobit si vzhled aplikace na světlý, tmavý, nebo uživatelsky systémový režim a jednoduchý text s popisem layout připojení pinů na Adafruit radič.



Obrázek 9. Vlevo screenshot ukazující barvy aplikace po vybrání Světlého režimu. Uprostřed po vybrání tmavého. Vpravo můžeme přepsat defaultní úhly nebo je obnovit do původního nastavení.

5 DISKUZE

Výsledkem práce je funkční prototyp systému pro ovládání pětiosého robotického ramene pomocí mobilní aplikace. V praxi se potvrdilo, že kombinace mikrokontroléru Arduino, Bluetooth modulu HC-05 a mobilní aplikace ve Flutteru představuje použitelný způsob bezdrátového ovládání robotického mechanismu v podmínkách školního projektu. Zvolené řešení odpovídá typickému přístupu u hobby a výukových konstrukcí, kde se klade důraz na dostupnost komponent, rychlou realizaci a ověření funkčnosti.

Z pohledu uživatelského ovládání se jako klíčové ukázalo vytvoření přehledného rozhraní pro řízení jednotlivých os a současně doplnění bezpečnostních omezení. Praktická zkušenost ukázala, že robotické rameno je citlivé na nevhodné kombinace poloh kloubů, což může vést ke kolizím nebo zvýšenému mechanickému namáhání. Proto byly do aplikace implementovány bezpečnostní zámky (lock/unlock) a bezpečnostní limity pohybu pro jednotlivé osy. Některé limity jsou dynamické, protože závisí na aktuální poloze jiné osy (např. maximální úhel pro SHOULDER podle polohy ELBOW). Tento přístup se osvědčil, protože snižuje riziko poškození konstrukce a zároveň zůstává pro uživatele srozumitelný, zejména díky vizuálním indikátorům bezpečného rozsahu přímo u sliderů.

Významným poznatkem z praktické realizace bylo, že u Bluetooth řízení nestačí pouze posílat aktuální hodnoty ze sliderů na zařízení. Při rychlém posouvání sliderů vzniká velké množství změn, které mohou zahltnit komunikaci nebo způsobit neplynulé a nestabilní chování zařízení. V průběhu vývoje se ukázalo, že bez omezení frekvence odesílání příkazů docházelo k problémům, kdy servo reagovalo nepředvídatelně a v některých situacích „vyjíždělo“ mimo očekávaný pohyb nebo přestávalo správně reagovat. Proto bylo nutné implementovat mechanismus debounce (odeslání příkazu až po krátké pauze) a doplnit ochranu proti duplicitním příkazům. Tím se snížilo zatížení přenosu, zlepšila se plynulost pohybu a zvýšila se stabilita ovládání.

Další částí projektu byla implementace automatizovaných sekvencí pohybu. Sekvenční režim se v praxi ukázal jako užitečné rozšíření, protože umožňuje opakovat předem definované pohyby bez ručního nastavování jednotlivých os. Pro definici sekvencí byl zvolen jednoduchý textový formát, který se snadno vytváří i upravuje. Současně se ale potvrdilo, že je nutné provádět validaci vstupních dat a poskytovat uživateli jasné chybové hlášení, aby nedocházelo k nečekanému chování při chybně zadané sekvenci.

Mezi nejčastější komplikace během vývoje patřily problémy se spuštěním projektu v důsledku nekompatibilních verzí závislostí. Konkrétně se jednalo o ladění kompatibility balíčku ``flutter_bluetooth_serial`` s verzí Flutteru a o nastavení správných verzí Android build nástrojů (např. Gradle). Tyto problémy se projevovaly zejména během spouštění a debugování aplikace, kdy bylo nutné upravit konfiguraci projektu tak, aby bylo možné aplikaci stabilně sestavit a spustit. Tato zkušenost ukazuje, že u mobilního vývoje je vedle samotného kódu důležitá i správná konfigurace vývojového prostředí a verzí nástrojů.

Z hlediska limitů práce je třeba zmínit, že výsledný prototyp je určen především pro demonstraci a výukové použití. Přesnost a opakovatelnost pohybu je ovlivněna vlastnostmi použitých servomotorů, mechanickými vřely konstrukce a také kvalitou napájení. Systém je navíc založen na přímém řízení jednotlivých úhlů serv, nikoliv na řízení koncového bodu pomocí inverzní kinematiky. Pro cíle této práce je však zvolený přístup vhodný, protože je přehledný, snadno pochopitelný a umožňuje praktické ověření principů řízení víceosého mechanismu.

Pro další rozvoj lze doporučit několik směrů. Prvním je rozšíření mobilní aplikace o další platformy, zejména o iOS, což může být omezeno podporou Bluetooth Classic na dané platformě. Druhým směrem je doplnění senzorů a zpětné vazby (např. informace o skutečné poloze, zatížení nebo chybovém stavu), která by mohla být odesílána ze zařízení zpět do aplikace. Třetím směrem je rozšíření práce se sekvencemi, například o jejich editaci přímo v aplikaci nebo o možnost „nahrávání“ pohybu a automatického vytvoření sekvence.

Celkově lze shrnout, že hlavní cíle práce byly splněny. Byl vytvořen funkční prototyp propojující mechanickou konstrukci robotického ramene, řízení přes Arduino, Bluetooth komunikaci a mobilní aplikaci. Praktická realizace zároveň ukázala, které části jsou pro stabilní použití klíčové: bezpečnostní limity pohybu, blokace konfliktů během sekvencí a řízení frekvence přenosu příkazů.

6 ZÁVĚR

Cílem práce bylo navrhnout a realizovat systém pro ovládání robotického ramene pomocí mobilního telefonu, a to jak v manuálním režimu, tak pomocí automatizovaných sekvencí. Tento cíl byl splněn vytvořením funkčního projektu, který propojuje robotické rameno řízené mikrokontrolerem Arduino s mobilní aplikací vytvořenou ve frameworku Flutter a bezdrátovou komunikací přes Bluetooth modul HC-05.

V rámci práce byla provedena fyzická realizace robotického ramene, zapojení elektronických komponent a implementace firmwaru pro zpracování řídicích příkazů. Byla vytvořena mobilní aplikace umožňující připojení přes Bluetooth, ovládání pěti os pomocí sliderů, úpravu výchozích hodnot v nastavení a spuštění automatizovaných sekvencí definovaných v textovém souboru. Důležitou součástí výsledného řešení jsou bezpečnostní zámky a limity pohybu, které snižují riziko nevhodných poloh a mechanických kolizí.

Výsledky práce ukazují, že pro stabilní řízení přes Bluetooth je nutné řešit nejen samotný přenos příkazů, ale také jejich frekvenci. Implementace mechanismu debounce a zamezení duplicitních příkazů přispěly k plynulejšímu chování a spolehlivějším reakcím robotického ramene.

Navržené řešení představuje funkční základ, který lze dále rozšiřovat. Do budoucna je možné uvažovat například o podpoře dalších platforem (např. iOS), o doplnění senzorů a zpětné vazby ze zařízení do aplikace, případně o rozšíření editoru sekvencí přímo v aplikaci.

7 CITOVANÁ LITERATURA

- [1] A. článku, „Stupěň volnosti,“ Wikipedie, 2 2 2007. [Online]. Available: https://cs.wikipedia.org/wiki/Stupe%C5%88_volnosti. [Přístup získán 16 3 2026].
- [2] A. článku, „Jednočipový počítač,“ Wikipedie, 1 12 2006. [Online]. Available: https://cs.wikipedia.org/wiki/Jedno%C4%8Dipov%C3%BD_po%C4%8D%C3%ADta%C4%8D. [Přístup získán 16 3 2026].
- [3] A. článku, „Servomotor,“ Wikipedie, 27 9 2005. [Online]. Available: <https://cs.wikipedia.org/w/index.php?title=Servomotor&action=history&dir=prev>. [Přístup získán 16 3 2026].
- [4] edufolly, „flutter_bluetooth_serial,“ Github, Neuvedeno. [Online]. Available: https://github.com/edufolly/flutter_bluetooth_serial. [Přístup získán 16 3 2026].
- [5] BUILD SOME STUFF, „How I Built a 3D Printed Robot Arm from Scratch (Arduino Based),“ 23 Duben 2023. [Online]. Available: <https://www.youtube.com/watch?v=5toNqaGsGYs&t=62s>. [Přístup získán 14 Únor 2025].
- [6] BUILD SOME STUFF, „Link to CAD Files,“ 23 Duben 2023. [Online]. Available: <https://drive.google.com/drive/folders/1x4P8AquQILwCp8e5CiRJLfVJiJn2U4cF>. [Přístup získán 14 Únor 2025].
- [7] u. uploader, „material 1.0.1,“ pub.dev, 18 10 2025. [Online]. Available: <https://pub.dev/packages/material>. [Přístup získán 16 3 2026].
- [8] F. package, „services lib,“ Flutter, [Online]. Available: <https://api.flutter.dev/flutter/services/>. [Přístup získán 16 3 2026].
- [9] „get_storage_package,“ Dart, [Online]. Available: https://pub.dev/documentation/get_storage/latest/. [Přístup získán 16 3 2026].
- [10] Dart, „dart:convert,“ Dart, [Online]. Available: <https://dart.dev/libraries/dart-convert>. [Přístup získán 16 3 2026].
- [11] Dart, „dart typed data,“ Dart, [Online]. Available: https://api.dart.dev/dart-typed_data/. [Přístup získán 18 3 2026].
- [12] baseflow.com, „permission handler,“ pub.dev, 3 7 2025. [Online]. Available: https://pub.dev/packages/permission_handler. [Přístup získán 16 3 2026].
- [13] Dart, „Asynchronous programming: futures, async, await,“ Dart, [Online]. Available: <https://dart.dev/libraries/async/async-await>. [Přístup získán 16 3 2026].

- [14] miguelruivo.com, „file picker,“ pub.dev, 26 1 2026. [Online]. Available: https://pub.dev/packages/file_picker. [Přístup získán 16 3 2026].
- [15] Dart, „File class,“ Dart, [Online]. Available: <https://api.dart.dev/dart-io/File-class.html>. [Přístup získán 16 3 2026].

8 SEZNAM POUŽITÝCH ZKRATEK

SPP – Sériový portový profil

RFCOMM – Radio Frequency Communication

BLE – Bluetooth Low Energy

PWM – Pulzně šířková modulace

SDK – software development kit

IDE – integrované vývojové prostředí

UI – user interface

BT – Bluetooth

9 SEZNAM OBRÁZKŮ

Obrázek 1. Popis jednotlivých stupňů volností, vytvořeno ve Fusion360, Screenshot. Autor	16
Obrázek 2. Schéma zapojení veškeré elektroniky, vytvořeno ve fritzing. Schéma Autor	17
Obrázek 3. Vlevo první obrazovka po načtení aplikace, uprostřed po stisknutí tlačítka Vyhledat... snackbar oznámení že bluetooth není zapnutý. Vpravo úspěšné vyhledání zařízení a seznam s připojením. Screenshots Autor	23
Obrázek 4. Jedna stejná scrollovatelná obrazovka-manuální ovládání serv. Vlevo jde vidět na obrazovce obrázek. Vpravo jdou po scrollnutí vidět všechny slidery. Screenshots Autor	25
Obrázek 5. Vlevo varovné dialogové okno po stisknutí zámku. Vpravo jednotlivé odemčené slidery bez omezujících hranic. Screenshots Autor	26
Obrázek 6. Diagram algoritmu sekvence. Diagram Autor	28
Obrázek 7. První zleva je obrazovka po spuštění. Druhý zleva je rozkliknutý combobox s výběrem ukázkových defaultních pohybů a výběrem vlastního souboru. Třetí zleva je výběr souboru ze systémové aplikace po kliknutí na Vlastní soubor... z druhého Screenshotu zleva. Čtvrtý zleva ukazuje že soubor byl úspěšně vybrán a je i zaškrtnutý loop. Screenshots Autor	29
Obrázek 8. Vlevo je spuštěná sekvence po načtení a jdou vidět jednotlivé kroky. Vpravo když se sekvence zastaví nebo dokončí, Screenshots Autor	30
Obrázek 9. Vlevo screenshot ukazující barvy aplikace po vybrání Světlého režimu. Uprostřed po vybrání tmavého. Vpravo můžeme přepsat defaultní úhly nebo je obnovit do původního nastavení.	31

10 ODKAZY

Repositář se zdrojovými kódy a obrázky: <https://github.com/sykecek/Projekt/tree/main>

Prezentace video: <https://youtu.be/SXWCkut7aJQ>